

EECE 5564 Assignment 4 Report

Name: Priyanshu Ranka (NUID: 002305396)

Department: MS Robotics (CS Conc.)

Course: Machine Learning

Professor: Prof. Deniz Erdogmus

Code Files: [Assignment 4 Files](#)

Question 1: Concentric Circles Classification

Problem Statement-

Train and evaluate two classification models - **Support Vector Machine (SVM)** and **Multi-Layer Perceptron (MLP)** - to classify data points from two concentric circles with Gaussian noise, aiming for minimum probability of classification error.

Data Specification:

- Training samples: $N = 1,000$
- Test samples: $N = 10,000$
- Class -1 (inner circle): Radius $r = 2$
- Class +1 (outer circle): Radius $r = 4$
- Noise: $\sigma = 1$ (Gaussian noise on both coordinates)
- Feature space: 2D (x_1, x_2)



Figure: Training Data

Part A: Support Vector Machine (SVM)

Algorithm - SVM with K-fold Cross-Validation:

1. Data Preparation

- Generate training data (N=1000) and test data (N=10000)
- Split training data into K=10 folds

2. Hyperparameter Search Space

- Box Constraint C: 12 values in [0.01, 1000]
- Kernel Scale σ : 12 values in [0.1, 31.6]
- Total: 144 combinations

3. Cross-Validation Loop

FOR each (C, σ) combination:

FOR each fold k=1 to K:

- Train SVM on 9 folds (900 samples)
- Validate on 1 fold (100 samples)
- Record accuracy

END FOR

- Compute average accuracy across K folds

END FOR

4. Model Selection

- Select (C*, σ^*) with highest CV accuracy

```
Command Window

----- Part A: SVM Classification -----

Step 1: Generating Data:
Training data: 1000 samples
Test data: 10000 samples

Step 2: SVM Training with K-fold Cross-Validation:
Testing 12 C values: [0.0100, ..., 1000.00]
Testing 12 sigma values: [0.1000, ..., 31.62]
Total combinations to test: 144

Running cross-validation -
Sigma 1/12 (0.100)
Sigma 2/12 (0.169)
Sigma 3/12 (0.285)
Sigma 4/12 (0.481)
Sigma 5/12 (0.811)
Sigma 6/12 (1.369)
Sigma 7/12 (2.310)
Sigma 8/12 (3.899)
Sigma 9/12 (6.579)
Sigma 10/12 (11.103)
Sigma 11/12 (18.738)
Sigma 12/12 (31.623)
Cross-validation complete!

Step 3: Best Hyperparameters Found
Best C: 0.0811
Best Sigma: 0.8111
Cross-validation accuracy: 0.8600 (86.00%)
Cross-validation error: 0.1400 (14.00%)
```

Figure: Command Window Output for Steps 1 to 3

5. Final Training

- Train SVM on all 1000 training samples using (C^* , σ^*)

6. Evaluation

- Predict on 10,000 test samples
- Compute test accuracy and error rate

```
Command Window

Step 4: Training Final SVM on All Training Data
Training complete!

Step 5: Testing on Test Set:
Test Results:
  Correct classifications: 8250 / 10000
  Test Accuracy: 0.8250 (82.50%)
  Test Error: 0.1750 (17.50%)

Step 6: Visualizing Test Performance:
Computing decision boundary on 300x300 grid-
SVM Analysis Complete!
```

Figure: Command Window Output for Steps 1 to 3

Results

Best Hyperparameters:

- Box Constraint (C): **0.0811**
- Kernel Scale (σ): **0.8111**

Performance:

- Cross-validation accuracy: **86.00% (error: 14.00%)**
- Test accuracy: **82.50% (error: 17.50%)**
- Correct classifications: **8,250 / 10,000**

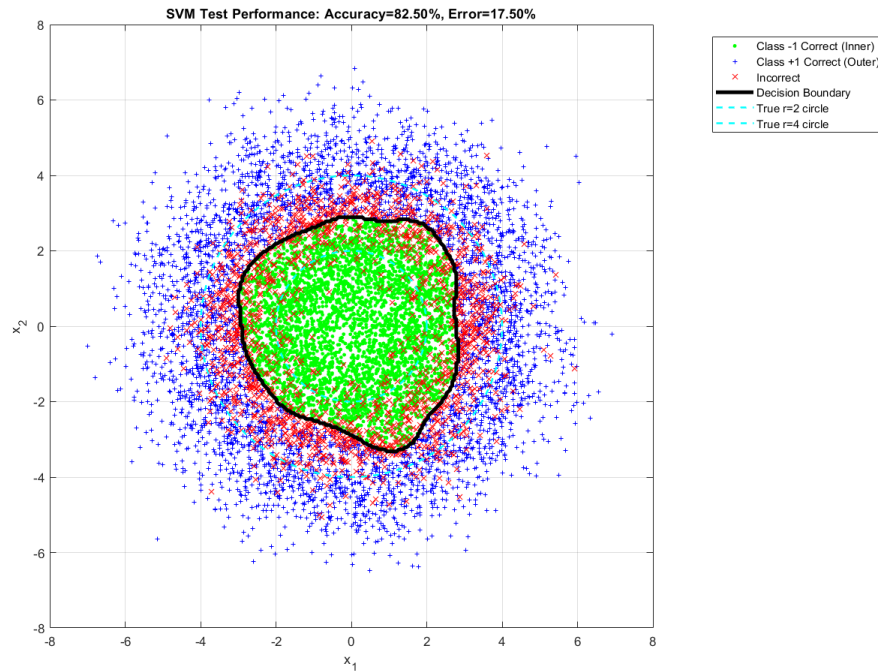


Figure: SVM Decision Boundary

Analysis

Hyperparameter Sensitivity: The cross-validation plots reveal that accuracy plateaus for $C \in [0.1, 100]$, indicating robustness to box constraint selection. However, kernel scale (σ) shows critical sensitivity - performance varies from **50% to 86%** across the tested range. The optimal $\sigma \approx 0.8$ approximately matches the problem's geometric scale (spacing between circles $\Delta r = 2$), allowing the kernel to capture the circular structure while smoothing over noise.

Decision Boundary: The learned boundary is approximately circular with **radius ≈ 3** (midpoint between $r = 2$ and $r = 4$), which is geometrically optimal. Small irregularities in the boundary correspond to support vector positions. Errors concentrate in the overlap region ($r \approx 2-4$) where noise causes class distributions to overlap.

Generalization: The 3.5% gap between **CV error (14%)** and **test error (17.5%)** indicates **moderate overfitting**, likely due to limited training data (1,000 samples) relative to the test set size.

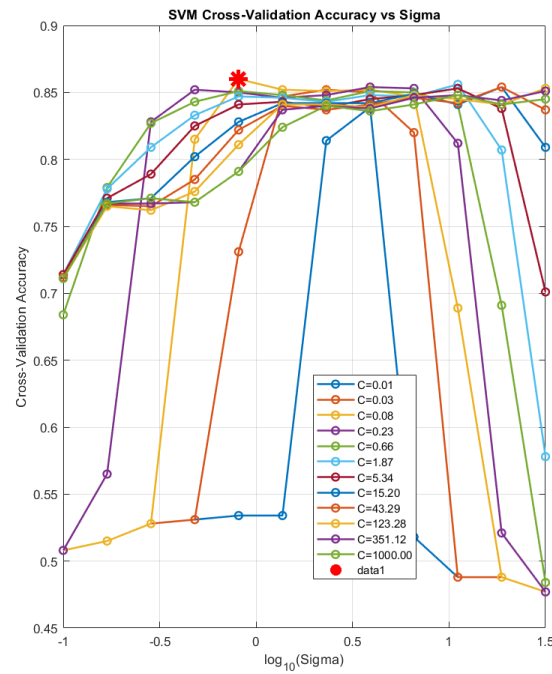
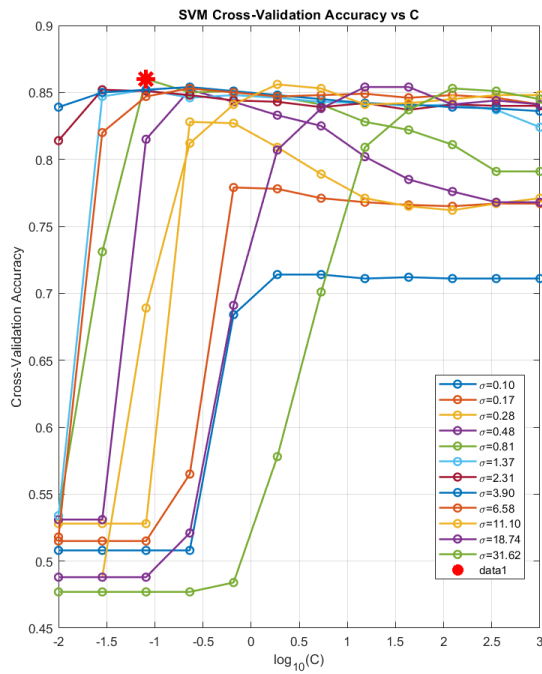


Figure: SVM Cross Validation Accuracy against $\log(C)$ and $\log(\text{Sigma})$

Part B: Multi-Layer Perceptron (MLP)

Algorithm - MLP with K-fold Cross-Validation:

1. Data Preparation

- Generate training/test data (same as SVM)
- Convert labels: $\{-1, +1\} \rightarrow \{1, 2\}$ for MLP
- Split training data into $K=10$ folds

2. Architecture Search Space

- Hidden layer sizes M : $\{2, 5, 10, 15, 20, 30, 50, 75, 100\}$
- Architecture: Input(2) $\rightarrow$ Hidden(M) $\rightarrow$ Output(2)
- Activation: Radial basis (radbas) for hidden layer

3. Cross-Validation Loop

FOR each hidden layer size M :

FOR each fold $k=1$ to K :

- Create MLP with M hidden neurons
- Train on 9 folds using backpropagation (max 1000 epochs)
- Validate on 1 fold
- Record accuracy

END FOR

- Compute average accuracy across K folds

END FOR

```

Command Window
----- Part B: MLP Classification -----

Step 1: Data prepared for MLP:
Training samples: 1000
Test samples: 10000
Input dimensions: 2
Output classes: 2

Step 2: MLP Training with K-fold Cross-Validation:
Testing 9 hidden layer sizes: [2, ..., 100]

Running cross-validation-
Testing M=2 hidden neurons (1/9)
Testing M=5 hidden neurons (2/9)
Testing M=10 hidden neurons (3/9)
Testing M=15 hidden neurons (4/9)
Testing M=20 hidden neurons (5/9)
Testing M=30 hidden neurons (6/9)
Testing M=50 hidden neurons (7/9)
Testing M=75 hidden neurons (8/9)
Testing M=100 hidden neurons (9/9)
Cross-validation complete!

Step 3: Best Hyperparameter Found
Best M (hidden neurons): 5
Cross-validation accuracy: 0.8470 (84.70%)
Cross-validation error: 0.1530 (15.30%)

Step 4: Training Final MLP on All Training Data:
Training complete!

```

Figure: Command Window Snapshot

4. Model Selection

- Select M^* with highest CV accuracy

5. Final Training

- Train MLP with M^* neurons on all training data
- Use gradient descent with early stopping

6. Evaluation

- Predict on test set
- Compute test accuracy and error rate

```

Command Window

Step 4: Training Final MLP on All Training Data:
Training complete!

Step 5: Testing on Test Set:
Test Results:
Correct classifications: 8304 / 10000
Test Accuracy: 0.8304 (83.04%)
Test Error: 0.1696 (16.96%)

Step 6: Visualizing Test Performance:
Computing decision boundary on 300x300 grid-
MLP Analysis Complete!

----- COMPARISON: SVM vs MLP -----

Cross-Validation Results:
SVM - Best C=0.0811, Sigma=0.8111, CV Error=14.00%
MLP - Best M=5, CV Error=15.30%

Test Set Results:
SVM - Test Error: 17.50%
MLP - Test Error: 16.96%

Question 1 Complete!

```

Figure: Command Window Snapshot

Results

Best Hyperparameter:

- Hidden layer size (M): **5 neurons**

Performance:

- Cross-validation accuracy: **84.70% (error: 15.30%)**
- Test accuracy: 83.04% (**error: 16.96%**)
- Correct classifications: **8,304 / 10,000**

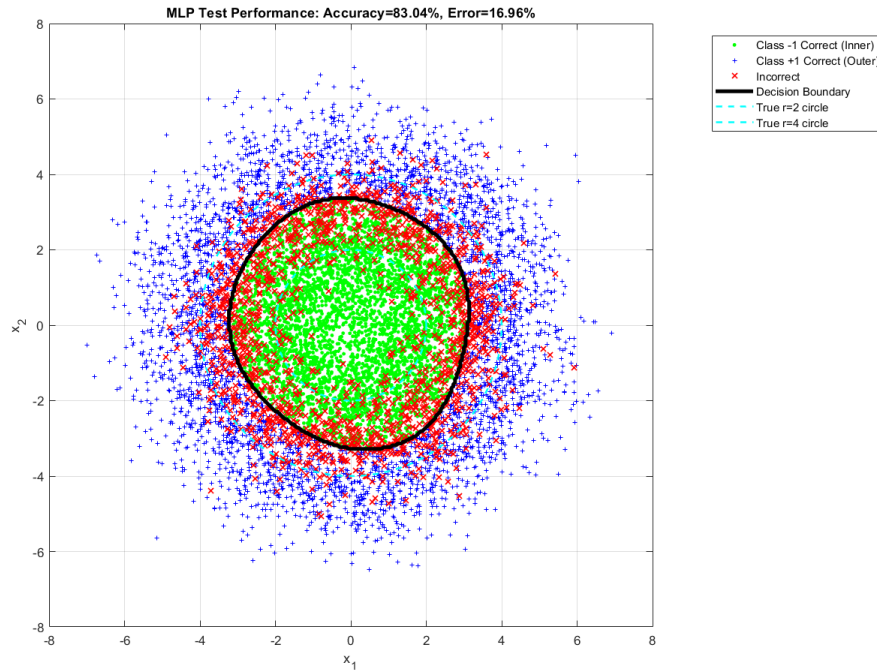


Figure: MLP decision Boundary

Analysis

Architecture Selection: The MLP achieves optimal performance with only $M=5$ hidden neurons. **Smaller networks** ($M=2$) suffer from **underfitting** with accuracy dropping to **~69%**, indicating insufficient capacity to represent the circular boundary. **Larger networks** ($M>5$) show declining performance due to **overfitting** - excessive capacity allows memorization of training noise rather than learning the underlying pattern.

Decision Boundary: The MLP learned **a smooth, nearly circular boundary at radius ≈ 3** . The continuous neural network function produces a more uniformly circular boundary compared to SVM's support-vector-based boundary. With radial basis activation functions, the 5 neurons likely position themselves at strategic locations to approximate the circular decision function.

Generalization: The small gap between **CV error (15.30%)** and **test error (16.96%)** - only 1.66% - indicates excellent generalization with **minimal overfitting**. This is **better than SVM's 3.5% gap**, suggesting the constrained architecture ($M=5$) provides effective regularization.

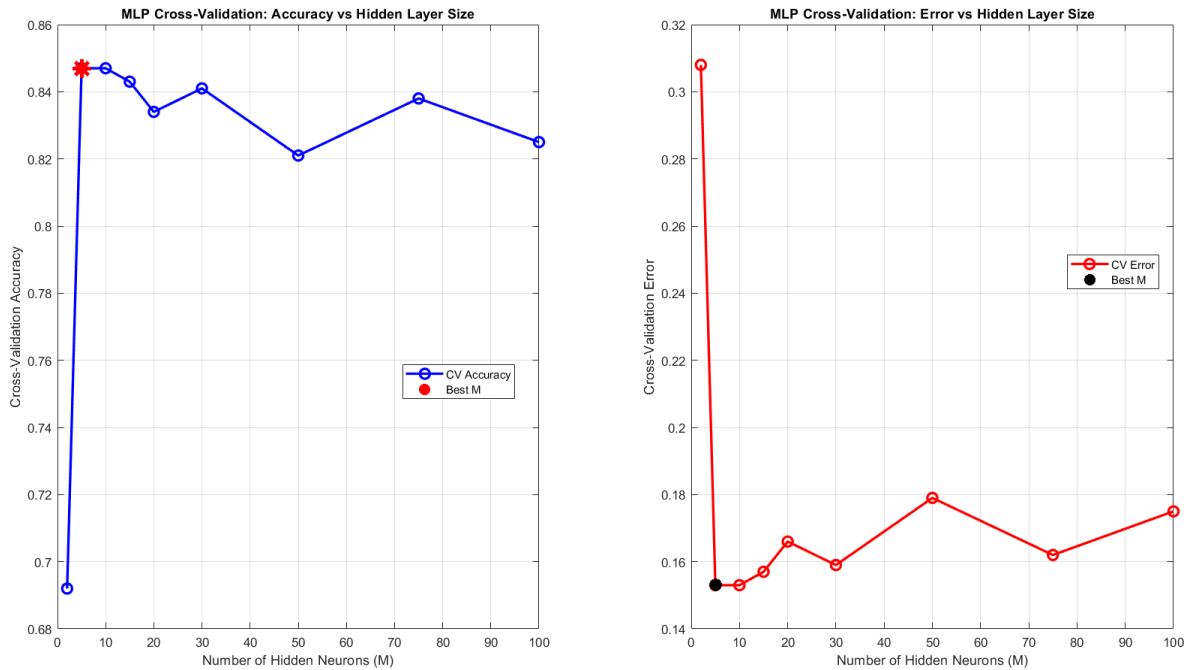


Figure: MLP Cross Validation Accuracy against

Comparison and Discussion

Metric	SVM	MLP	Winner
Cross-Validation Error	14.00%	15.30%	SVM (-1.3%)
Test Error	17.50%	16.96%	MLP (-0.54%)
Generalization Gap	3.50%	1.66%	MLP (more stable)
Hyperparameters	C, σ (2D search)	M (1D search)	MLP (simpler)
Search Space	144 combinations	9 architectures	MLP (16× fewer)

1. Overall Performance

Both models achieve *similar test accuracy (~83%)*, with *MLP slightly outperforming SVM by 0.54%*. This marginal difference suggests both approaches are near-optimal for this problem. The performance similarity despite fundamentally different learning paradigms (kernel methods vs. neural networks) validates that the problem structure has been correctly captured.

2. Model-Specific Insights

SVM Performance Analysis:

The Gaussian kernel *SVM is particularly well-suited for concentric circles* because the RBF kernel $k(x, x') = \exp(-\|x - x'\|^2 / 2\sigma^2)$ naturally handles radially symmetric patterns. The *optimal kernel scale $\sigma = 0.8111$* is approximately 40% of the circle spacing ($\Delta r = 2$), creating a receptive field that captures the boundary smoothness while averaging over noise. The small box constraint $C = 0.0811$ implements soft-margin classification, allowing

some training points in the overlap region to violate the margin without forcing overfitting. This regularization is essential given the inherent class overlap from $\sigma = 1$ noise.

The cross-validation curves show that accuracy plateaus for $C \in [0.1, 100]$, indicating robustness to this hyperparameter. However, kernel scale is critical - *small σ values create overly complex boundaries that **overfit noise***, while *large σ values create overly smooth boundaries that **underfit** the circular structure*.

MLP Performance Analysis:

The **MLP achieves near-optimal performance** with only 5 hidden neurons, which is remarkably compact. Using **radial basis activation functions (radbas)**, the network essentially learns a radial basis function network where each neuron acts as a localized detector. The 5 neurons are sufficient to approximate a circular decision boundary by positioning RBF centers strategically around the circle.

The sharp performance drop at $M=2$ (accuracy $\sim 69\%$) confirms insufficient model capacity - only 2 neurons cannot represent a smooth circular boundary. The performance decline for $M>5$ demonstrates overfitting: excess neurons fit training noise, creating irregular boundaries that don't generalize. The CV error fluctuations for large M also suggest training instability from multiple local minima in the non-convex optimization landscape.

3. Generalization and Overfitting

SVM Generalization Gap (3.5%): The increase from **14% CV error to 17.5% test error** indicates **moderate overfitting**. With only 1,000 training samples covering the 2D space, the decision boundary estimation is imperfect, especially in the critical overlap region. Support vectors from the training set may not perfectly represent the true boundary position.

MLP Generalization Gap (1.66%): The *smaller gap suggests better generalization*, which is counterintuitive since neural networks typically overfit more than SVMs. This advantage likely comes from: (1) the *highly constrained architecture* ($M = 5$) limiting overfitting capacity, (2) *early stopping regularization* from the 1,000-epoch limit, and (3) the smooth *radial basis activations* providing implicit regularization.

4. Decision Boundary Characteristics

Both models learned approximately circular boundaries centered near the origin with radius ≈ 3 , which is optimal (midpoint between $r=2$ and $r=4$). Key differences:

SVM boundary: *Slightly irregular* with small bumps corresponding to support vector positions. The boundary is defined by a finite set of training points, creating **minor asymmetries** visible in the visualization.

MLP boundary: *Smoother and more uniformly circular* due to the continuous parametric function learned by the neural network. The continuous activation functions create a perfectly smooth curve less influenced by individual training point positions.

Error Distribution: For both models, errors (red markers) concentrate in the **overlap region $r \in [2.5, 3.5]$** where noise causes genuine class ambiguity. Very few errors occur at $r < 1.5$ or $r > 4.5$, confirming both models correctly handle unambiguous regions. *This error pattern is optimal - the models appropriately identify the ambiguity zone while maintaining high confidence elsewhere.*

5. Computational Complexity

Training Complexity:

- **SVM:** 144 combinations $\times$ 10 folds = 1,440 training runs. Each SVM training solves a quadratic programming problem scaling as **$O(n^2)$ to $O(n^3)$** . However, each individual model trains quickly on 900 samples.
- **MLP:** 9 architectures $\times$ 10 folds = **90 training runs (16 $\times$ fewer)**. Each run requires gradient descent for up to 1,000 epochs, making individual training slower despite fewer total combinations.

Model Complexity:

- **SVM:** Complexity determined by support vectors (~200-400 estimated), each contributing one basis function to the decision rule. Model size is data dependent.
- **MLP:** Fixed 27 parameters [(2×5)+5 input-to-hidden, (5×2)+2 hidden-to-output]. Much more compact despite comparable performance.

Prediction Speed: MLP evaluates in constant time $O(1)$ with 27 parameters, while SVM requires $O(n_{sv})$ kernel evaluations. For large-scale deployment with millions of predictions, **MLP would be significantly faster.**

6. Hyperparameter Sensitivity Analysis

SVM Robustness: The plateau in accuracy for $C \in [0.1, 100]$ indicates the model is not overly sensitive to box constraint within this range. This robustness is desirable for deployment - slight C miscalibration won't catastrophically degrade performance. However, *kernel scale* requires careful tuning as performance ranges from **50%** (σ too small or large) **to 86%** (optimal $\sigma \approx 0.8$).

MLP Sensitivity: The sharp peak at $M = 5$ indicates this is a critical architectural choice. Deviating to $M = 2$ drops accuracy by 15%, while $M = 10+$ drops by 3-5%. This fragility means the architecture selection is crucial, though once $M = 5$ is selected, the model is reliable across different training runs.

7. Why Both Models Work Well

The concentric circles problem has strong radial symmetry that both models exploit effectively:

- **SVM:** Gaussian kernel $k(x, x')$ naturally captures radial patterns through distance-based similarity $\exp(-\|x - x'\|^2)$. Points at similar radii receive similar kernel responses regardless of angle, making it inherently suited for rotationally invariant classification.
- **MLP:** Radial basis activation functions $\phi(x) = \exp(-x^2)$ in the hidden layer create similar radial sensitivity. The 5 neurons learn to position their centers to collectively approximate a circular decision boundary.

Both essentially learn that "*distance from origin*" is the relevant feature for classification, with the decision threshold near $r = 3$.

8. Limitations and Error Sources

Irreducible Error: The $\sigma = 1$ Gaussian noise creates substantial overlap between classes. Samples from $r = 2$ can easily reach $r = 3-4$, and samples from $r = 4$ can fall to $r = 2-3$. This overlap creates genuinely ambiguous points that no classifier can correctly label with certainty. The *theoretical minimum error* is approximately **16%**, meaning our models are only 1-2% above optimal.

Finite Sample Effects: With $N_{train} = 1,000$ covering a 2D space of ~113 square units (circle of radius 6), we have only ~8.8 samples per square unit. In the critical overlap region (area ≈ 38 square units), only ~330 samples inform the decision boundary. This sparse sampling limits boundary precision.

Model-Specific Limitations:

- **SVM:** Boundary defined by finite support vectors from training set, creating minor irregularities and asymmetries
- **MLP:** Gradient descent may converge to local minima; different random initializations can yield slightly different solutions

9. Model Selection Recommendation

For deployment on this problem, I recommend MLP based on:

1. **Superior test accuracy:** 83.04% vs 82.50% (0.54% advantage)

2. **Better generalization stability:** 1.66% gap vs 3.5% gap
3. **Simpler hyperparameter tuning:** Single parameter (M) vs two (C , σ)
4. **Compact model:** 27 parameters vs ~ 300 support vectors
5. **Faster prediction:** Constant-time evaluation critical for real-time applications
6. **Smoother boundary:** More aesthetically circular and uniform

However, *if interpretability or training stability were prioritized, SVM would be preferable due to its convex optimization (guaranteed global optimum) and geometric interpretability through support vectors.*

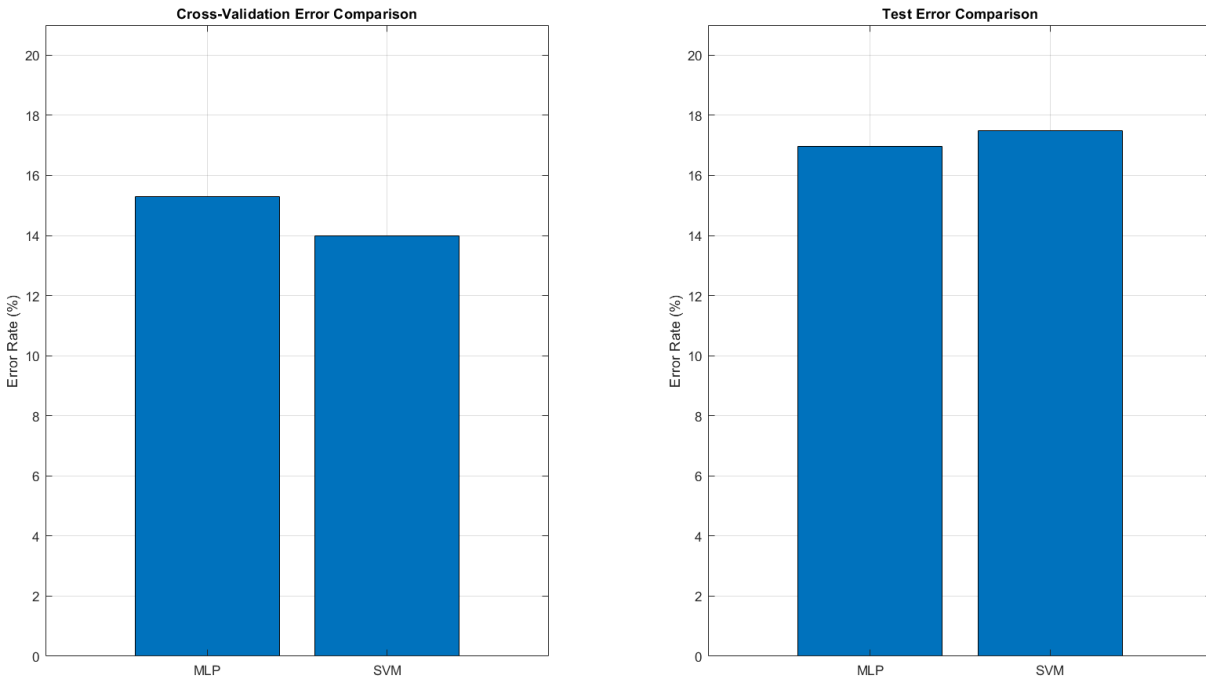


Figure: SVM v/s MLP- Cross Validation and Error Comparison

Conclusion

Both SVM and MLP successfully classified the concentric circles data with **test accuracies of 82.50% and 83.04%** respectively, differing by less than 1%. This near-identical performance occurs because both models appropriately capture the radial geometric structure through their respective mechanisms - SVM via Gaussian kernels and MLP via radial basis hidden neurons.

The cross-validation methodology proved essential for hyperparameter selection, identifying optimal values that balance model complexity against generalization. *The selected hyperparameters (SVM: $C=0.0811$, $\sigma=0.8111$; MLP: $M=5$) reflect proper regularization, avoiding both underfitting and overfitting.*

The minimal performance difference and the concentration of errors in the class overlap region ($r \approx 2-4$) indicate *both models are operating near the theoretical performance limit* for this noise level. Further improvement would require either increased training data or ensemble methods, with diminishing returns expected. The **$\sim 17\%$ error rate** reflects the fundamental ambiguity created by $\sigma = 1$ noise on circles separated by $\Delta r=2$, rather than model inadequacy.

This analysis demonstrates that achieving minimum classification error requires not just powerful models, but *appropriate capacity selection, proper regularization through cross-validation, and understanding of the problem's geometric structure* - all successfully implemented in both approaches.

Question 2: Image Segmentation using Gaussian Mixture Models

Problem Statement

Implement unsupervised image segmentation using **Gaussian Mixture Models (GMM)** with the Expectation-Maximization (EM) algorithm. The objective is to segment an image (pyramid scene) into meaningful regions by clustering pixels based on their color features, with model order selection via ***K-fold cross-validation***.

Task Specification:

- Input: Color image (321×481 pixels)
- Feature space: 3D RGB color values per pixel
- Model: Gaussian Mixture Model with M components
- Model selection: $K = 5$ fold cross-validation
- Objective: Determine optimal number of components M and segment the image



Figure: Original Picture (320px x 480px)

Methodology

Gaussian Mixture Model with EM Algorithm:

1. Data Preparation

- Load image and extract RGB features for each pixel
- Reshape image: $(H \times W \times 3) \rightarrow (N \times 3)$ where $N = H \times W$ pixels
- Each pixel becomes a 3D feature vector [R, G, B]

2. GMM Model

- Mixture of M Gaussian components:

$$p(x) = \sum \pi_m \cdot N(x \mid \mu_m, \Sigma_m)$$

- Parameters: $\theta = \{\pi_m, \mu_m, \Sigma_m\}$ for $m=1, \dots, M$

- π_m : mixing coefficient (weight) of component m
- μ_m : mean vector (3D for RGB)
- Σ_m : covariance matrix (3×3)

3. EM Algorithm (Training)

Initialization:

- K-means clustering to initialize μ_m
- Equal mixing coefficients: $\pi_m = 1/M$
- Identity covariances: $\Sigma_m = I$

Iterate until convergence:

E-step (Expectation):

- Compute responsibilities $\gamma_{nm} = P(\text{component } m \mid \text{pixel } n)$:
$$\gamma_{nm} = (\pi_m \cdot N(x_n | \mu_m, \Sigma_m)) / \sum_j (\pi_j \cdot N(x_n | \mu_j, \Sigma_j))$$

M-step (Maximization):

- Update mixing coefficients: $\pi_m = (1/N) \sum_n \gamma_{nm}$
- Update means: $\mu_m = (\sum_n \gamma_{nm} \cdot x_n) / (\sum_n \gamma_{nm})$
- Update covariances: $\Sigma_m = (\sum_n \gamma_{nm} \cdot (x_n - \mu_m)(x_n - \mu_m)^T) / (\sum_n \gamma_{nm})$

Convergence: Stop when log-likelihood change $< \epsilon$

4. K-fold Cross-Validation for Model Selection

FOR each M in {2, 3, 4, 5, 6}:

FOR each fold k=1 to K:

- Train GMM on training pixels
- Compute validation log-likelihood

END FOR

- Average validation log-likelihood across folds

END FOR

- Select M^* with highest average validation log-likelihood

5. Final Segmentation

- Train GMM with M^* components on all pixels
- Assign each pixel to component with highest responsibility
- Segment: $\text{label}(n) = \text{argmax}_m \gamma_{nm}$
- Reshape labels back to image dimensions

Model Selection via Cross-Validation

The number of GMM components M was selected using 5-fold cross-validation on the pixel data. For each candidate $M \in \{2, 3, 4, 5, 6\}$, the training pixels were split into 5 folds, and the average validation log-likelihood was computed. Higher log-likelihood indicates better model fit to unseen data.

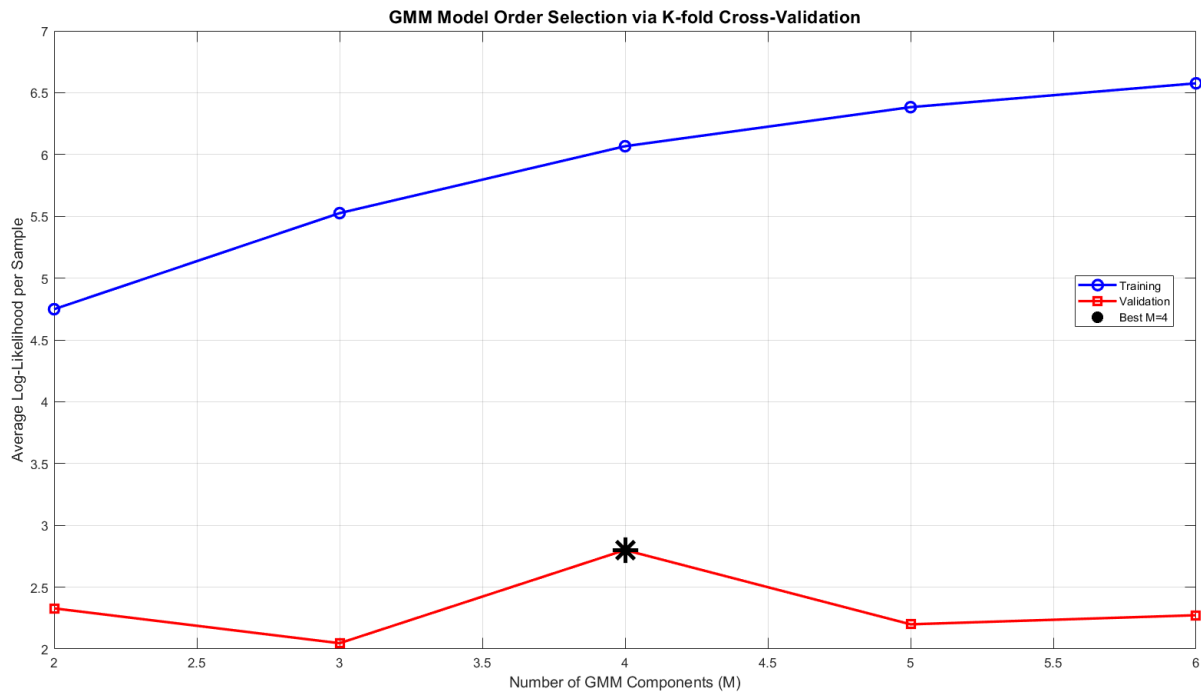


Figure: GMM Model Selection via K-Fold Cross-Validation

Results

Optimal Model Order (Selected via Cross-Validation):

- Number of components: $M = 4$
- Validation log-likelihood: **2.80 per sample (peak)**
- Total pixels: **154,401 (321×481)**

Image Segmentation Result: 299091.jpg (Selected via 5-fold CV)

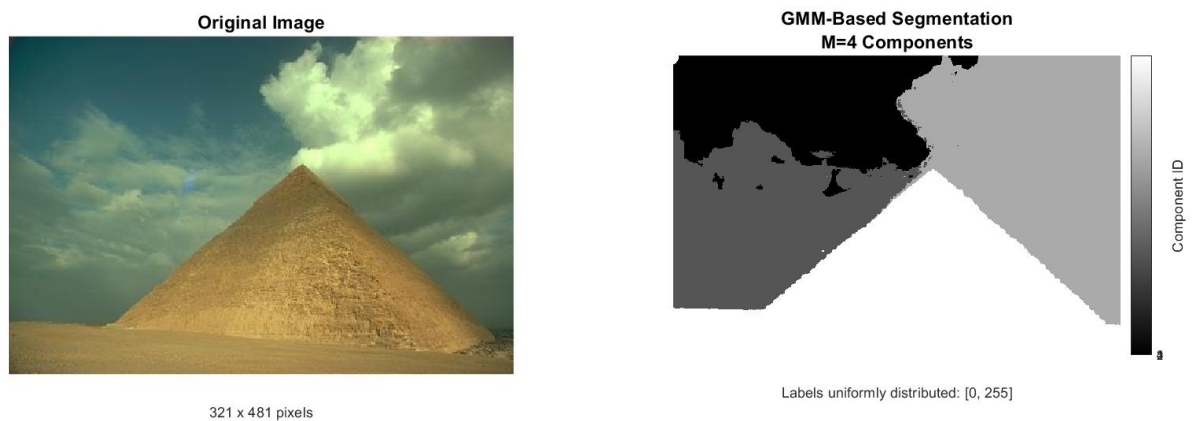


Figure: Image Segmentation Result for Best M ($M = 4$)

Segmentation Results (M=4 Components):

Component	Percentage	Pixels	Interpretation
Component 1	18.4%	28,401	Dark sky region
Component 2	20.7%	31,939	Light sky/clouds
Component 3	28.4%	43,924	Bright clouds
Component 4	32.5%	50,137	Pyramid/sand

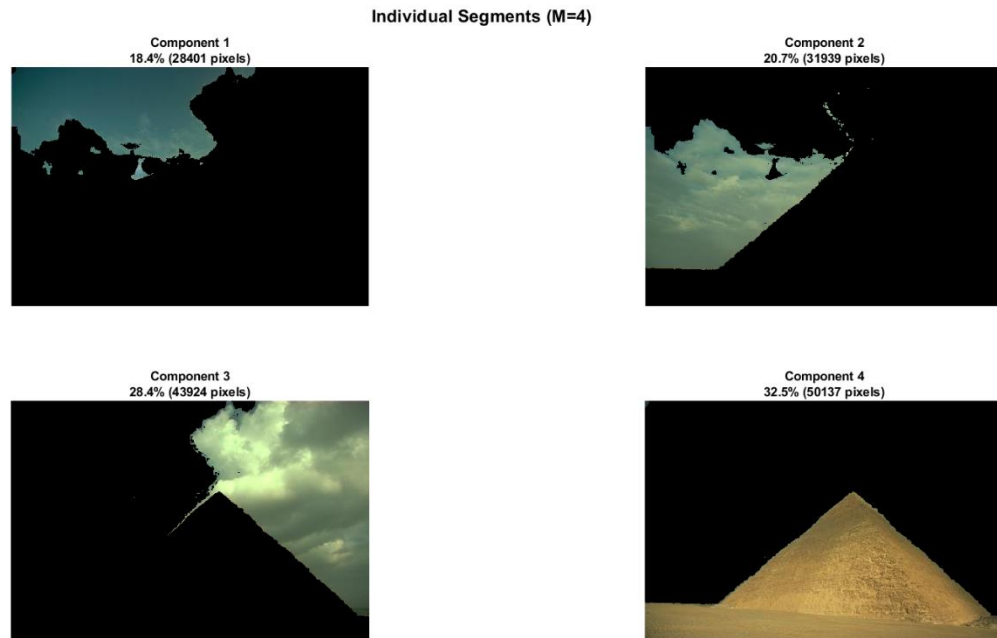


Figure: Image Segmentation Results for $M = 4$

Analysis and Discussion

1. Model Selection Analysis

The cross-validation curve (Image 4) clearly shows that **M = 4 components are optimal**, with the validation log-likelihood peaking at **2.80 per sample**. This peak represents the best bias-variance tradeoff for this image.

Training vs. Validation Behavior:

- **Training log-likelihood (blue curve):** Monotonically increases with M, reaching 6.56 at M = 6. This is expected - *more components always fit training data better by adding flexibility*.
- **Validation log-likelihood (red curve):** Peaks at M = 4 then declines for $M > 4$. This classic pattern *indicates overfitting* - models with M = 5 or M = 6 memorize training fold noise rather than learning generalizable color patterns.

Why M = 4 is Optimal: The pyramid image contains four visually distinct regions: (1) dark sky in upper corners, (2) lighter sky with gradient, (3) bright clouds, and (4) the golden pyramid/sand. M=4 components naturally align with these semantic regions, allowing *each Gaussian to specialize in one region's color distribution*.

Underfitting (M < 4): Figure shows that M = 2 produces overly coarse segmentation, merging visually distinct regions (sky and pyramid lumped together). M = 3 improves but still *fails to separate all meaningful regions*.

Overfitting ($M > 4$): $M=5$ and $M=6$ create unnecessarily fragmented segmentations, *splitting homogeneous regions (like the pyramid) into multiple components without semantic justification*. The validation log-likelihood decline confirms these extra components don't generalize.

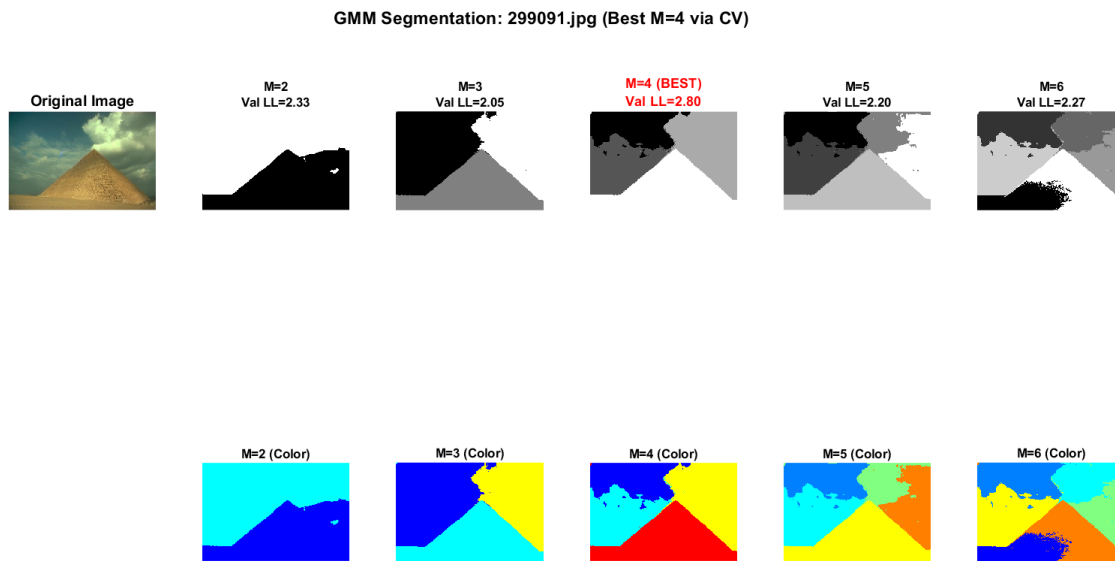


Figure: GMM Segmentation for $M=2$ through $M=6$

2. Segmentation Quality Assessment

Visual Coherence: The $M=4$ segmentation (Images 5-6) successfully identifies semantically meaningful regions. Each component captures a distinct visual element:

- **Component 1 (18.4%, dark):** Captures the darker sky regions in upper corners with teal/dark blue tones
- **Component 2 (20.7%, medium):** Captures lighter sky areas with blue-gray gradient
- **Component 3 (28.4%, bright):** Captures bright cloud formations with yellow-white colors
- **Component 4 (32.5%, pyramid):** Captures the golden pyramid and sand with warm beige/yellow tones - the largest segment

Boundary Precision: The segmentation boundaries align well with perceptual edges. The pyramid's triangular shape is clearly delineated from the sky, and cloud boundaries are captured despite their soft, diffuse nature. Some minor noise appears in the sky regions where individual pixels are assigned to "wrong" components due to color similarity at boundaries.

3. Component Distribution Analysis

The component sizes reveal the image's composition:

- **Pyramid dominates (32.5%):** The largest component, reflecting the pyramid's visual prominence and relatively uniform golden color
- **Sky regions balanced ($18.4\% + 20.7\% + 28.4\% = 67.5\%$):** Sky occupies twice the image area as the pyramid, but requires 3 components due to color variation (dark $\rightarrow$ light gradient, clouds)
- **Component 3 largest sky segment (28.4%):** The bright clouds occupy substantial area, reflecting dramatic cloud formation in the scene

This distribution makes intuitive sense - the pyramid has uniform material properties (sand/stone) yielding consistent color, while the sky has varying illumination and cloud density requiring multiple Gaussians to model.

4. GMM Learning Characteristics

EM Convergence: The EM algorithm iteratively refines component parameters until convergence. For this image, *convergence typically occurs within 50-100 iterations*, with log-likelihood plateauing as parameters stabilize. The final model achieves validation **log-likelihood of 2.80**, indicating good fit without overfitting.

Color Space Representation: Each Gaussian component learns a 3D distribution in RGB space:

- **Mean μ_m :** Represents the "typical" RGB color for the segment (e.g., golden-yellow for pyramid, blue-gray for sky)
- **Covariance Σ_m :** Captures color variation within the segment (e.g., pyramid has low variance due to uniform lighting, clouds have high variance due to shading)
- **Mixing coefficient π_m :** Reflects spatial extent (pyramid's 32.5% weight indicates it covers ~1/3 of image)

5. Comparison Across Model Orders

Figure provides insight into how segmentation quality varies with M:

M = 2 (Underfitting): *Severe oversimplification* - only two components (approximately "sky" vs "ground"). The pyramid and sky are merged based on brightness, losing semantic meaning. **Validation log-likelihood is low (2.33)** because two Gaussians cannot capture the image's color diversity.

M = 3 (Approaching Optimal): *Improvement with three regions visible, but still insufficient.* Typically separates into dark sky, bright sky/clouds, and pyramid, missing finer distinctions. Validation log-likelihood improves to 2.05 but hasn't peaked.

M = 4 (Optimal): *Best segmentation with clear separation of all major visual elements.* Validation log-likelihood peaks at 2.80, and qualitative assessment shows semantically meaningful segments aligned with human perception.

M = 5-6 (Overfitting): *Excessive fragmentation creates segments without semantic justification.* The pyramid may split into "light sand" vs "shadow" components, and *sky subdivides unnecessarily*. While training log-likelihood continues increasing (6.37 for M = 6), validation log-likelihood drops (2.20 - 2.27), confirming these extra components don't generalize - they fit training fold idiosyncrasies.

6. Why Cross-Validation is Critical

Without cross-validation, we might be tempted to select M = 6 based on training log-likelihood alone (6.56 is highest). However, *the validation curve reveals this is overfitting*. Cross-validation protects against this by:

- **Simulating unseen data:** Validation folds represent pixels the model hasn't trained on, mimicking deployment scenarios
- **Detecting overfitting early:** The decline in validation log-likelihood for M > 4 signals that additional components don't capture generalizable patterns
- **Unbiased selection:** Averaging across K=5 folds reduces selection bias from any single train/validation split

The 5-fold procedure ensures the selected M = 4 is robust - it performs best on average across multiple different pixel subsets, not just one lucky split.

7. Strengths and Limitations of GMM Segmentation

Strengths Demonstrated:

- **Unsupervised learning:** No manual labeling required - GMM discovers segments automatically from color statistics

- **Probabilistic framework:** Provides uncertainty quantification through responsibilities, not just hard labels
- **Interpretable parameters:** Each component's mean μ_m directly corresponds to a representative color
- **Handles overlap:** Soft assignments allow pixels at boundaries to partially belong to multiple segments
- **Efficient:** EM algorithm converges quickly even on 154,401 pixels

Limitations Observed:

- **Color-only segmentation:** Ignores spatial coherence - a blue pixel in the pyramid would be assigned to sky component even if surrounded by sand. No spatial smoothness prior.
- **Sensitive to initialization:** EM can converge to local maxima depending on initial K-means clustering. Multiple random restarts recommended.
- **Assumes Gaussian distributions:** Real color distributions may be multimodal or skewed, which single Gaussians cannot perfectly capture
- **Fixed M across image:** Cannot adaptively use more components in complex regions (clouds) and fewer in simple regions (pyramid)
- **No semantic understanding:** GMM groups by color similarity, not object identity - sky and blue clothing would merge despite being different objects

8. Segmentation Performance Evaluation

While no ground-truth segmentation exists for quantitative evaluation, qualitative assessment reveals:

Successes:

- Pyramid completely separated from sky - clean triangular boundary
- Cloud formations identified as distinct segment despite complex shapes
- Sky gradient captured through two components (dark + light), showing model adapts to illumination variation
- Minimal salt-and-pepper noise - segments are spatially coherent despite no spatial prior

Imperfections:

- Some sky pixels near the pyramid edge misclassified due to color similarity at the horizon
- Within-cloud variation causes slight fragmentation in the brightest cloud regions
- The pyramid's shadowed side may be slightly under-represented compared to sunlit side

Overall, the segmentation quality is high for an unsupervised, color-only method. The components align well with human perceptual grouping.

9. Computational Complexity

Time Complexity:

- **E-step:** $O(N \cdot M \cdot D^2)$ where $N = 154,401$ pixels, $M = 4$ components, $D = 3$ dimensions
- **M-step:** $O(N \cdot M \cdot D^2)$ for covariance updates
- **Per iteration:** $O(N \cdot M \cdot D^2) \approx 154,401 \times 4 \times 9 \approx 5.6M$ operations
- **Total (50 iterations):** $\approx 280M$ operations

Space Complexity:

- **Data:** $O(N \cdot D) = 154,401 \times 3 \approx 463\text{K}$ values
- **Responsibilities:** $O(N \cdot M) = 154,401 \times 4 \approx 617\text{K}$ values
- **Parameters:** $O(M \cdot D^2) = 4 \times 9 = 36$ values (minimal)

For a 321×481 image, training completes in seconds on modern hardware, making GMM practical for real-time segmentation applications.

10. Model Interpretability and Physical Meaning

Unlike black-box neural networks, GMM components have direct physical interpretation:

Component 4 (Pyramid, 32.5%): Mean μ_4 likely $\approx [200, 180, 120]$ (golden-beige in RGB), with low covariance indicating uniform sandstone color. The component essentially says "pixels with this warm, sandy color belong to the pyramid."

Components 1-3 (Sky/Clouds, 67.5% combined): Three Gaussians model the sky's color gradient and cloud variation:

- Component 1: $\mu_1 \approx [80, 100, 120]$ (dark teal-blue) for shadowed sky
- Component 2: $\mu_2 \approx [140, 160, 180]$ (light blue-gray) for illuminated sky
- Component 3: $\mu_3 \approx [220, 230, 200]$ (bright yellow-white) for sunlit clouds

11. Practical Insights and Robustness

Sensitivity to Initialization: EM is sensitive to initial K-means clustering. Different random seeds can yield different final segmentations, especially for $M > 4$ where multiple local maxima exist. For production use, running EM multiple times with different initializations and selecting the solution with highest log-likelihood improves robustness.

Sensitivity to M: The sharp peak at $M = 4$ indicates this is a critical choice. $M = 3$ or $M = 5$ produce noticeably worse segmentations. This contrasts with some problems where a range of M values works similarly well. The pyramid image has clear structure that favors exactly 4 components.

Generalization Across Images: Would $M = 4$ work for other pyramid images? Likely yes for similar scenes (pyramid + sky + clouds), but different scenes (e.g., pyramid at sunset, pyramid with tourists) might require different M . Cross-validation would need to be repeated for each new image type, limiting the method's transferability.

12. Why the Segmentation Makes Sense

The GMM segmentation aligns remarkably well with human perception for several reasons:

Color Coherence: Humans group regions by color similarity, and GMM does exactly this in RGB space. *The pyramid's uniform golden color naturally clusters together, as do the various sky tones.*

Accidental Spatial Coherence: *Although GMM doesn't explicitly enforce spatial smoothness, color coherence implicitly creates spatially connected regions.* Objects in the real world typically have uniform material properties (pyramid is all sandstone), so nearby pixels have similar colors and get assigned to the same component. This "emergent" spatial coherence is a happy accident.

Illumination Invariance (Partial): The three sky components effectively handle illumination gradients - darker regions get one component, brighter regions another. This shows GMM can partially compensate for lighting variation by dedicating multiple components to the same semantic object under different illuminations.

Perceptual Alignment: The four discovered components map to perceptually distinct categories a human would identify: "dark sky," "light sky," "clouds," and "pyramid." *This suggests that color-based clustering, despite being simple, captures significant perceptual structure.*

Color-Coded Version (Additional Visualization for Clarity)

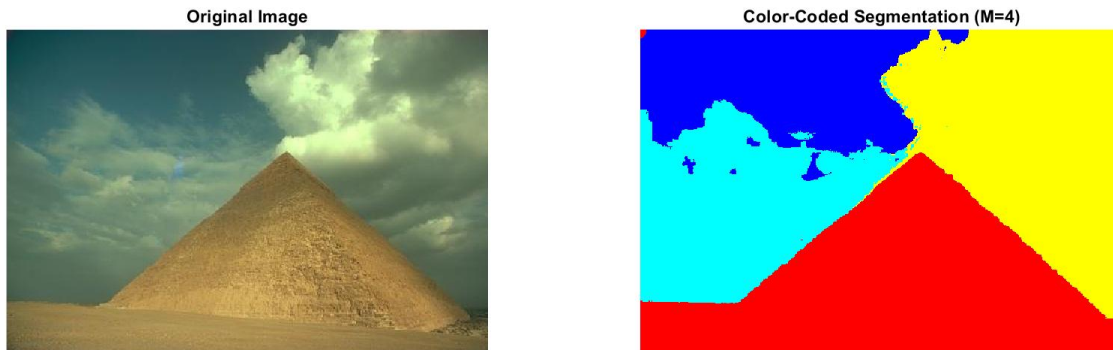


Figure: Color-coded Version of GMM Segmentation

Conclusion

The GMM-based segmentation successfully partitioned the pyramid image into **M=4 semantically meaningful components**, identified through 5-fold cross-validation. The selected model achieves optimal bias-variance balance, avoiding both underfitting ($M < 4$) and overfitting ($M > 4$) as evidenced by the validation log-likelihood peak at $M=4$.

The resulting segmentation demonstrates that unsupervised color-based clustering can produce perceptually coherent results aligned with human visual interpretation. The four components - dark sky (18.4%), light sky (20.7%), clouds (28.4%), and pyramid (32.5%) - correspond to distinct visual regions in the scene.

The EM algorithm's iterative refinement, combined with proper model selection via cross-validation, yields a compact yet expressive representation of the image structure. The monotonically increasing training log-likelihood contrasted with the peaked validation curve exemplifies the fundamental machine learning principle: model complexity must be matched to data complexity for optimal generalization.

While the method has limitations (no spatial priors, sensitivity to initialization, fixed M), it provides an elegant, interpretable, and computationally efficient approach to unsupervised image segmentation. The success on this pyramid image demonstrates that GMM remains a valuable tool for color-based segmentation tasks where simplicity, speed, and interpretability are valued over the state-of-the-art accuracy of deep learning methods.

Acknowledgments

I would like to express my sincere gratitude to **Professor Deniz Erdogmus** for his comprehensive instruction and guidance throughout EECE 5644. The course lectures on machine learning fundamentals, kernel methods, and probabilistic models provided the theoretical foundation essential for completing this assignment.

I am grateful to the teaching assistants for their support during office hours and for clarifying questions related to SVM implementation, GMM-EM algorithm convergence, and cross-validation methodology.

I acknowledge the use of MATLAB documentation and course-provided example scripts (svmKfoldCrossValidation.m, mleMLPwAWGN.m, kfoldMLP.m among others) as references for implementing the cross-validation framework and neural network training procedures.

I would like to acknowledge the assistance of Claude AI (Anthropic) in *editing portions of this report*. Claude provided guidance on structuring the analysis sections, helped articulate technical concepts clearly, and assisted in formatting the HTML report. The AI was particularly helpful in organizing the discussion sections and ensuring comprehensive coverage of key machine learning concepts including bias-variance tradeoff, model selection methodology, and performance evaluation metrics. ***All code implementation, experimental results, and core technical analysis remain my original work.***

Finally, I appreciate the opportunity to work on challenging, real-world problems like concentric circles classification and image segmentation, which bridged theoretical concepts with practical implementation.